System Verification and Validation Plan for Physiotherapy Companion

Team 11, technically functional
Matthew Huynh
Cieran Diebolt
Vaisnavi Shanthamoorthy
Maham Siddiqui
Eman Ashraf

April 3, 2026

Revision History

Name(s)	Date	Notes
Maham, Cieran and Eman	Oct 27, 2025	Added initial first draft of the VnV Plan
Date 2	1.1	Notes

Contents

List of Tables

1 Symbols, Abbreviations, and Acronyms

symbol	description
T	Test

Refer to Section 4 in the SRS document for a table of many more symbols, abbreviations, acronyms, and definitions.

This document aims to outline the verification and validation that the Physiotherapy Companion app will go through during development. The document begins with a brief description as to what the software is, what it is meant to achieve, and some relevant information before explaining in detail the standards and metrics that will be used to evaluate the system.

2 General Information

2.1 Summary

The software focused on in this document is the Physiotherapy Companion application, built for smart devices with camera access to assess a patient undergoing physiotherapy on their form during assigned physiotherapy exercises.

The application will provide support and guidance for the exercise and make real-time suggestions to improve the user's form using a video-feed. It will also provide overviews of previous exercises to allow users to see progress and consistency, assisting in forming healthy habits towards recovery and reinforce confidence.

The application will also support a more administrative function for physiotherapists, allowing them to view exercise summaries for patients assigned to them. Additionally, physiotherapists may leave comments on summaries to ensure consistent and timely usage of the application for healthy exercising.

2.2 Objectives

The objectives of the testing expanded upon within this document focus on general system usability, the security of user profiles and video-feed data, and the verification of stakeholder requirements.

Usage of the system should be easily explainable for first-time users with simple dialogue popups and limited visible guiding from one function to the next. Additionally, for the video-feed portion of the application's usage, the user should be reasonably guided through the exercise and process to set them up for successful exercise execution and form recommendation acceptance.

User profile and video-feed data should be securely stored, whether locally or within a remote database accessed by the system. User data security has

become an increasingly sensitive topic within the software industry of late and users may question how their data is being stored and utilized. Users should be aware of the usage of their data and the system should ensure unauthorized access outside of the communicated plan is never possible.

Meeting stakeholder requirements, whether they be data restrictive or process flow in nature, is what will validate the core features of the system. If users do not find value in the expected usage expected from the system, then, ultimately, the system has failed to perform its primary goal. Ensuring stakeholder requirements and feedback are incorporated into the system as it is developed, both initially and onward, will ensure the system is both wanted and usable to the intended audience.

Some out-of-scope objectives include performance and user-interface quality. In this sense, quality means the overall refinement and how polished the interface looks. The implementation should not inhibit usability, but the design need not be too meticulous and visually pleasing.

For performance, the system should respond in a timely manner that does not frustrate a user's ability to perform the main activities of the system. Designing an optimized system which may use minimal amounts of resources is not one of the current objectives of the system. These things could both be expanded into and developed, but for now are considered out of scope.

2.3 Extras

The extras set out for the system consist of a Design Thinking Document and a User Instructional Video.

The Design Thinking Document aims to explain the process behind, and work done towards, the final design of the system. This document explains the overall flow of processing through the system and the interfacing elements, external libraries utilized, and more. The document flow is iterative, as is the design process, and aims to communicate the entire design process of the system.

The User Instructional Video aims to communicate the method of usage of the system to the primary stakeholders. The video will demonstrate the usage of the dashboard and analytics display, the video-feed form recommendation function, as well as an explanation of the overall feedback provided for each completed exercise. AThe instructions will be patient-centred. For the physiotherapist view the patient list, comment features, and analytical display function will be explained.

2.4 Relevant Documentation

- Problem Statement and Goals: Ashraf et al. [2025d]
- Development Plan: Ashraf et al. [2025b]
- System Requirements Specification: Ashraf et al. [2025e]
- Hazard Analysis: Ashraf et al. [2025c]
- User Instructional Video: Ashraf et al. [2025f]
- Design Thinking Document: Ashraf et al. [2025a]

All of the above document are important artifacts for the implementation of the system. These documents outline how the implementation will be approached and what the implementation will follow.

More documents to come here for external libraries such as OpenCV, pytest, Python Extension, and more when it is determined which libraries and external software will be used.

3 Plan

This section will focus on the methods to verify and validate artifacts of the project. The methods for each facet of the system will be listed below, and additional information including possible questionnaire questions can be found in the appendix at the end of this document. The section will start primarily with verification and move to validation at the end.

3.1 Verification and Validation Team

See *Table 1: Verification and Validation Team.*

3.2 SRS Verification

To verify the SRS, several approaches will be used across different sections of the document. In general, the document will receive verification through feedback from 4G06 teaching assistants, peer-review feedback from another design team, and a check in with a project advisor to ensure the document

Role	Member(s)	Description
Test Development	Maham, Eman	Development of tests for the system and system units based on functional and non-functional requirements listed within the SRS document.
Test Implementation	Cieran	Implementation of a test suite for feasibly software testable requirements, developed based on the tests made by the test development team. Also assurance of software coverage by the suite.
Test Execution	Matthew	Running and verification of the test suite.
Requirements Verification	Vaisnavi	Verification and tracking of requirements to ensure each has been met and tested effectively.
Stakeholder Validation	All	Validation with stakeholders and external users to ensure the system developed performs as users expect; focus on usability.

Table 1: Verification and Validation Team

represents the correct motivation for the project. This check-in will be a self review by the advisor followed by a scheduled meeting to present any probable modifications to the requirements. Based on feedback from the advisor, issues will be created and handled.

To verify the functional requirements within the SRS, the above will be used in addition to software checks for functionality and a requirements checklist which will be updated upon checking functional requirements within the implementation. For verification of non-functional requirements within the SRS, in addition to the general checks, there will be a questionnaire of the software's usage and another checklist with a more flexible scaling system for the NFRs.

3.3 Design Verification

Verification of the design will rely on feedback from 4G06 teaching assistants, peer-review feedback provided by colleagues on another design team, as well as advisor and stakeholder inquiries. The feedback from teaching assistants and peer-review feedback will be created as issues within our repository and handled accordingly. For advisor and stakeholder inquiry, there will be software handouts along with a questionnaire which is expanded on more at the end of this section.

3.4 Verification and Validation Plan Verification

This document will also be reviewed by the teaching assistants and provided peer-review by another design team, creating issues within the GitHub repository to be dealt with accordingly. To ensure feedback is properly actioned, the following process is followed:

- Feedback is collected during TA and peer review sessions
- Each comment is converted into a GitHub issue
- Issues are assigned to a responsible team member
- Changes are implemented and committed with reference to the issue
- Updated sections are reviewed again before finalizing

An issue is considered resolved once the required changes have been implemented, reviewed by at least one additional team member, and the corresponding GitHub issue has been closed. In addition, the test suite derived from this document will be verified via code coverage on the software. For validation, the final outcome of each test will be discussed in the meeting with the project advisor to ensure the system is doing what it should in each test case.

3.5 Implementation Verification

The implementation verification plan ensures that the system aligns with the System Requirements Specification (SRS) through both dynamic and non-dynamic testing methodsDynamic testing is conducted through unit testing, integration testing, and system-level validation. Unit tests are implemented for both the frontend and backend components to verify core functionality such as authentication, data handling, and computer vision analysis. Integration testing is used to validate interactions between system components, including Firebase authentication, Firestore database operations, and backend API endpoints such as `/process_frame` and `/precheck_frame`. Performance testing is also carried out to evaluate response time during exercise recording ($\leq$ 2 seconds), system stability under repeated usage, and handling of real-time data processing. Non-dynamic (static) testing is used to evaluate code and design quality without executing the system. This includes structured peer code reviews on all pull requests to assess correctness, readability, and adherence to best practices. Code walkthroughs are performed for critical modules such as authentication and computer vision analysis to help identify potential issues early. In addition, basic static analysis tools such as `pylint` are used where applicable to check for common coding issues in the backend. Document reviews are also conducted, with issues tracked and resolved through GitHub. Automated test suites are executed during development to detect regressions and ensure that all existing functionality remains intact. GitHub Actions is used to support automated workflows such as builds and checks on pull requests, helping maintain project stability. Manual testing is also performed for real-time features, including camera-based pose detection and user interaction workflows that cannot be fully automated.

3.6 Automated Testing and Verification Tools

Unit Testing Framework: The project uses standard testing tools for different parts of the system. `pytest` is used for the Python backend to validate API endpoints, data processing logic, and computer vision functionality. For the frontend, tests are written using `Jest` to verify UI behavior and component interactions. These tools were selected due to their reliability, documentation support, and compatibility with the development environment.

Code Coverage Tools and Plan: Code coverage is used to measure how much of the system is exercised by the implemented test cases.

- **Python:** `pytest-cov` is used alongside `pytest` to track code coverage for backend components. This includes measuring which parts of the API logic and processing functions are executed during testing.
- **TypeScript:** `Jest` provides built-in coverage reporting for frontend components, tracking execution of functions and user interaction flows.

The goal is to achieve meaningful coverage across both codebases, focusing on critical functionality such as authentication, data handling, and core processing logic. Coverage results are used during development to identify untested areas and improve test completeness.

Linters and Static Analysis: To maintain code quality, basic static analysis tools such as `pylint` are used for backend code where applicable. In addition, structured peer code reviews and walkthroughs are conducted on pull requests to ensure readability, correctness, and adherence to best practices.

Automated Workflows: GitHub Actions is used to support automated workflows such as builds and checks on pull requests. While full test automation through CI is not currently implemented, these workflows help ensure that the project remains stable as changes are introduced.

3.7 Software Validation

For software validation, to ensure the correct system was designed to solve the problems outlined within the problem statement document, there will be system walkthroughs and interviews with stakeholders and the project advisor. These interviews will assess time to use the systems functions, understanding of the system and how it processes data, as well as a general

acceptance of the system. The interview will end with questions outlined in the appendix of this document. (Section 6.2)

4 System testing

The tests outlined below have been divided into 3 key areas:

1. **User registration suite:** the tests listed here have been designed for the purpose of testing the system's handling of the registration process.
2. **PT specific testing:** These tests have been tailored to test the functionalities available to the physiotherapists.
3. **Patient focused testing:** The results from this will give insight into how well the system accomplishes its intended purpose.

Note: Automatic tests will be run using a script. See Appendix for an example template of script. The use of 'display' indicates both audio and visual formats are provided.

4.1 User registration suite

This section contains: user_acc_create, invalid_format_create, cannot_verify_creds, acc_exists.

Table 2: Software Test Cases for User Account Creation

Test Case ID	**user_acc_create**
Control	Automatic
Rationale	The system must be able to create and store a new account.
State	User does not have an account on the application.
Input	Govt id (name, date of birth), license number (PT) OR invite code (patient), email, password (will be set at the end of the registration process)
Output	"Account created successfully" displayed on page and user is successfully added to the user database

How the test will be performed	The test will be conducted with the use of a script (see template in Appendix)
Test Case ID	**invalid_format_create**
Control	Automatic
Rationale	The data input by the user has to be in the correct format for our system to accept it as valid.
State	User does not have an account on the application.
Input	Incorrect format of license number/invite code OR incorrect format of date of birth, email, password (will be set at the end of the registration process)
Output	System displays 'incorrect format of one or more inputs'
How the test will be performed	The test will be conducted with the use of a script (see template in Appendix)
Test Case ID	**cannot_verify_creds**
Control	Automatic
Rationale	Successful verification for both types of users is needed to use the application
State	User does not have an account on the application.
Input	Incorrect license number/mismatch of license number and govt id OR incorrect format of invite code, email, password (will be set at the end of the registration process)
Output	System displays 'License number cannot verified' OR 'invalid invite code'
How the test will be performed	The test will be conducted with the use of a script (see template in Appendix)
Test Case ID	**acc_exists**
Control	Automatic
Rationale	New users should not already have an account on the system
State	Account already exists on system

Input	Govt id (name, date of birth), license number/invite code, email, password
Output	'Account already exists. Wrong password. Try again.'
How the test will be performed	The test will be conducted with the use of a script (see template in Appendix)

4.2 PT focused testing

This section contains: PT_sends_invite, PT_adjusts_exercise, PT_deletes_patient_acc.

Table 3: Software Test Cases for Physical Therapist (PT) Functions

Test Case ID	**PT_sends_invite**
Control	Automatic
Rationale	A patient can only sign up if they have an invite code from the PT.
State	Patient does not have an account on the application while PT does.
Input	PT adds patient's information and contact method (where the code will be sent).
Output	An 'Invite sent' message is displayed on the screen
How the test will be performed	The test will be conducted with the use of a script (see template in Appendix)
Test Case ID	**PT_adjusts_exercise**
Control	Automatic
Rationale	The system allows the PT to obtain an overview of patient performance and adjust different parameters accordingly (eg. number of repetitions).
State	PT and patient have an account on application, PT has patient on their patient list

Input	PT accesses the patient's profile page and adds/modifies exercise
Output	Patient exercise criteria is modified.
How the test will be performed	The test will be conducted with the use of a script (see template in Appendix)
Test Case ID	**PT_deletes_patient_acc**
Control	Automatic
Rationale	This ensures PTs control over patients' accounts. PT can remove patient(s) profiles from the application.
State	PT and patient have account on application, PT has the patient on their patient list
Input	PT requests a specific patient's account deletion
Output	Account deleted from user database and PT patient list. Patient is unable to log in ('Account not found').
How the test will be performed	The test will be conducted with the use of a script (see template in Appendix)

4.3 Patient focused testing

This section contains: system_exercise_overview, system_record_exercise, system_performs_correctly, video_store, video_retrieval, post_ex_patient_feedback, incorrect_form_adjustment

Table 4: Software Test Cases for Exercise System and Additional Functions

Test Case ID	**system_exercise_overview**
Control	Automatic
Rationale	In order to record the patient performing the exercise, the system has to be able to provide instructions to the patient on how to perform it.

State	Patient is logged into the application and has it open on their device. Patient has selected the exercise they want to perform on their exercise list.
Input	Patient clicks on exercise from exercise list.
Output	System displays an overview of the steps required for the exercise.
How the test will be performed	The test will be conducted with the use of a script (see template in Appendix)
Test Case ID	**system_record_exercise**
Control	Manual
Rationale	Patient performance of the exercise has to be successfully recorded to aid system analysis of its correctness.
State	Patient is currently logged in and has the application open on their device. Patient has selected the exercise they would like to perform from the exercise list.
Input	Patient clicks on the 'record' button
Output	Exercise successfully recorded, saved and ready for viewing
How the test will be performed	• Patient clicks on the 'exercise list' and selects the exercise they would like to perform • System directs patient to a screen with a live video stream of themselves • Patient clicks on the 'record' button • System records exercise
Test Case ID	**system_performs_correctly**
Control	Manual
Rationale	System has to effectively perform analysis on patient exercise form to provide feedback.
State	Patient is recording exercise
Input	System receives visual of patient performing exercise

Output	System provides feedback to patient (e.g. instructions for the next steps in the exercise)
How the test will be performed	• Patient performs exercise correctly • System monitors and analyzes exercise performance in real-time • System provides correct feedback
Test Case ID	**video_store**
Control	Automatic
Rationale	The system has to be able to store a recorded video of the patient performing the exercises.
State	Patient has performed and recorded exercise being performed.
Input	Patient clicks on 'Save exercise' button. *Note: The team has not decided whether to store the video locally or on the user database.*
Output	Video has been successfully saved and is ready for viewing.
How the test will be performed	The test will be conducted with the use of a script (see template in Appendix)
Test Case ID	**video_retrieval**
Control	Automatic
Rationale	The system has to be able to retrieve the most recent recorded video in storage (either locally or database).
State	Patient has performed and recorded exercise being performed
Input	Patient OR PT clicks on video stored (local or database) for viewing. *Note: The team has not decided whether to store the video locally or on the user database.*
Output	Video has been successfully retrieved and is ready for viewing.
How the test will be performed	The test will be conducted with the use of a script (see template in Appendix)
Test Case ID	**post_ex_patient_feedback**

Control	Automatic
Rationale	The system has to be able to store patient feedback of the performed exercise (e.g. the level of (perceived) difficulty of the exercise)
State	Patient has performed and recorded exercise being performed and system has successfully performed analysis of the performance
Input	Patient input into feedback form (See Appendix *Patient post-exercise feedback form*).
Output	Answers are stored successfully
How the test will be performed	The test will be conducted with the use of a script (see template in Appendix)
Test Case ID	**incorrect_form_adjustment**
Control	Manual
Rationale	The system has to be able to differentiate between correct and incorrect forms to provide accurate feedback.
State	Patient is performing the selected exercise
Input	Incorrect form during exercise performance
Output	Adjustment directives displayed to patient
How the test will be performed	• Patient performs exercise with incorrect form • System detects the incorrect posture/movement • System provides real-time adjustment instructions

5 Nonfunctional Requirements Evaluation

Table 5: Non-Functional Requirements Testing

Test Case ID	**ui_testing**
Type	Static
Rationale	Ensure user interface meets usability standards and stakeholder expectations
State	N/A
Input	User interface elements and survey responses
Output	Modified user interface based on feedback
How the test will be performed	• A survey will be given to users (see 'Stakeholders' in SRS for the type of users) to test different interface elements • See Appendix for survey questions • Based on the feedback received, the user interface will be modified in order to satisfy additional requirements
Test Case ID	**ex_int_speed**
Type	Automatic
Rationale	Ensure exercise interface loads within acceptable time limits for good user experience
State	User is logged in and has application open on their device
Input	User clicks on exercise interface
Output	Interface loads within 2 seconds
How the test will be performed	The test will use external tools (Postman) to measure API latency. In order to replicate real world conditions, an increasing amount of simulated user loads can be used. Statistical analysis will be performed on the collected results to provide a more accurate value.
Test Case ID	**fail_recovery**
Type	Dynamic

Rationale	The system must be able to restore its pre-failure state 95% of the time within 5 minutes
State	User is currently using the application on their device
Input	System failure (simulated)
Output	System restarts and recovers within 5 minutes
How the test will be performed	• A failure scenario will be simulated • Recovery time and success rate will be measured • System's state will be compared to pre-failure state to confirm if properly restored.
Test Case ID	**Interrupted_vid**
Type	Dynamic
Rationale	The system must be able to attempt resubmission of video after restoring internet connection
State	A video of the user performing the exercise is being uploaded
Input	Internet connection interruption during upload
Output	Video is successfully re-queued for upload to the system. Re-upload is resumed at the point of failure
How the test will be performed	• Network is disconnected during video upload • Connection is restored after interruption • The system detects interruption and queues video for re-upload • Confirm upload resumes from point of failure (not restarting) • Video integrity is confirmed after successful upload
Rationale	**PR-RFT 3, PR-RFT 4, PR-CR1, PR-SER 1** These requirements address advanced system capabilities beyond current testing scope

How the test will be performed	• These requirements are unable to be tested in a timely manner due to project constraints (time, money) • Testing deferred to future development phases • Requirements marked for re-evaluation in next project iteration

5.1 Traceability Between Test Cases and Requirements

Please refer to the following Microsoft Excel file: Traceability Matrix

6 Unit Test Description

6.1 Unit Tests

This section outlines the unit testing approach, scope, and representative test cases for each module. Detailed input/output data and executable tests are maintained in the project's testing repository.

6.2 Unit Testing Scope

All major modules developed for the system are within the scope of unit testing, including authentication, organization linking, exercise management, video capture, computer vision, reporting, data storage, and user interface components.

Third-party libraries and SDKs (e.g., video encoding, encryption frameworks) are out of scope for direct unit testing. These are verified indirectly through integration and acceptance testing. Modules responsible for UI styling and documentation display are of lower priority since their risk level is minimal; they will be verified primarily through automated UI snapshot tests.

Testing focuses on functional correctness, data integrity, and compliance with fitness criteria. Each module includes both **black box** (external behavior) and **white box** (internal logic) tests.

6.3 Tests for Functional Requirements

Most verification for the functional requirements will occur through automated unit tests. Each module below outlines representative tests that validate the implementation of key requirements.

6.3.1 Module 1: Authentication and Account Management

This module verifies that only authorized users can access protected features and that account creation, login, and security rules comply with FR 1–4 and SR AR 1–4.

Test ID: AUTH-UT-001
Type: Functional, Automatic
Initial State: System with no active user session.
Input: Attempt to access the dashboard without logging in.
Output: Access is denied with an authentication error message (HTTP 401 or equivalent).
Test Case Derivation: From FR 1 — users must log in before accessing exercises or dashboards.
How Test Will Be Performed: Run automated request simulation; verify the controller response and log entry.

Test ID: AUTH-UT-002
Type: Functional, Automatic
Initial State: Empty user database.
Input: Attempt to register with missing required fields or duplicate credentials.
Output: Registration is rejected; descriptive validation errors are returned.
Test Case Derivation: From FR 1.1 — all required fields must be completed, and credentials must be unique.
How Test Will Be Performed: Automated validation tests using mocked data submissions.

Test ID: AUTH-UT-003
Type: Functional, Automatic
Initial State: Active logged-in session.
Input: 60 minutes of inactivity simulated with timers.

Output: Session is automatically terminated; subsequent requests are rejected.
Test Case Derivation: From SR-AR 3 — system must log out inactive users.
How Test Will Be Performed: Use time mocking; verify session invalidation behavior.

6.3.2 Module 2: Organization Linking and Access Codes

This module ensures physiotherapists and patients connect securely under FR 1.2–1.3 and FR 2–3.

Test ID: ORG-UT-001
Type: Functional, Automatic
Initial State: Unverified physiotherapist profile.
Input: Submit incomplete license or clinic credentials.
Output: Verification denied; PT access blocked until all credentials provided.
Test Case Derivation: From FR 1.2 — PT access requires all four credential fields.
How Test Will Be Performed: Automated validation of registration form with missing inputs.

Test ID: ORG-UT-002
Type: Functional, Automatic
Initial State: Patient account without an organization link.
Input: Attempt to join with invalid and valid access codes.
Output: Invalid codes rejected; valid code links patient to exactly one PT and clinic.
Test Case Derivation: From FR 1.3 and FR 3 — patients can only join with a valid PT-issued code.
How Test Will Be Performed: Automated join requests tested against mock organization data.

6.3.3 Module 3: Exercise Program Management

Covers FR 5–6, FR 10–11, FR 22, and FR 27. Ensures exercises can be created, customized, and tracked properly.

Test ID: PROG-UT-001
Type: Functional, Automatic
Initial State: Exercise library initialized.
Input: Request for exercise list by an authorized patient.
Output: At least 10 knee exercises with instructions and media are returned.
Test Case Derivation: From FR 5 — the system must include a complete knee-exercise library.
How Test Will Be Performed: Automated content verification test querying the exercise repository.

Test ID: PROG-UT-002
Type: Functional, Automatic
Initial State: Existing exercise plan.
Input: PT updates sets, reps, and rest time.
Output: Plan updated immediately; change logged with timestamp.
Test Case Derivation: From FR 6 — PTs can customize patient exercises.
How Test Will Be Performed: Automated API test comparing new and old plan versions.

Test ID: PROG-UT-003
Type: Functional, Automatic
Initial State: PT sets weekly limits on exercise metrics.
Input: Patient attempts to exceed limit.
Output: System prevents action and displays a limit-exceeded message.
Test Case Derivation: From FR 27 — PT-defined daily/weekly limitations must be enforced.
How Test Will Be Performed: Simulate plan execution and validate limit enforcement.

6.3.4 Module 4: Video Capture and Review

Tests FR 7, FR 15, FR 26, and user-environment checks.

Test ID: CAP-UT-001
Type: Functional, Automatic
Initial State: Device camera available.
Input: Begin exercise recording.
Output: Video captured at ≥720p @ 30 fps; stored successfully for PT review.
Test Case Derivation: From FR 7 — video must meet minimum quality standards.
How Test Will Be Performed: Automated hardware simulation verifying metadata fields.

Test ID: CAP-UT-002
Type: Functional, Automatic
Initial State: Patient ready to record in a low-light environment.
Input: System performs pre-capture check.
Output: Warning prompts user to improve lighting before recording.
Test Case Derivation: From UHR-EOU 3 — app must alert user to poor conditions.
How Test Will Be Performed: Simulated light-level input producing prompt verification.

Test ID: CAP-UT-003
Type: Functional, Automatic
Initial State: Captured but incomplete exercise data.
Input: Submit out-of-frame video.
Output: System rejects submission, provides retry prompt with reason.
Test Case Derivation: From FR 26 — retries allowed for insufficient data.
How Test Will Be Performed: Mock analysis engine returning "insufficient data" flag.

6.3.5 Module 5: Computer Vision and Form Analysis

Validates FR 8, FR 12–14, FR 16, and related accuracy requirements.

Test ID: CV-UT-001
Type: Functional, Automatic

Initial State: Analysis model loaded.
Input: Test video with known joint angles.
Output: Angles computed within ±5°; range of motion within ±10%.
Test Case Derivation: From FR 12 and PR-PAR 1 — measurement accuracy thresholds.
How Test Will Be Performed: Automated comparison with expert-measured reference data.

Test ID: CV-UT-002
Type: Functional, Automatic
Initial State: Exercise attempt with poor form.
Input: Analyze deviation from model.
Output: "Incorrect form" message specifying deviation (e.g., insufficient bend) within 1 second.
Test Case Derivation: From FR 13–14 — system must identify and describe form issues.
How Test Will Be Performed: Simulated exercise data fed to analysis engine; timing measured.

Test ID: CV-UT-003
Type: Functional, Automatic
Initial State: Detected dangerous deviation.
Input: Critical form violation (>30° misalignment).
Output: Exercise halted; warning displayed.
Test Case Derivation: From PR-SCR 1 — prevent injury by halting unsafe movement.
How Test Will Be Performed: Inject synthetic "critical" deviation data; verify halt condition.

6.3.6 Module 6: Reporting and Dashboards

Confirms FR 17–20, FR 25, and FR 18 (PT visibility).

Test ID: RPT-UT-001
Type: Functional, Automatic
Initial State: Completed exercise session.

Input: Generate session report.
Output: Report includes completion rate, form score, pain level, and feedback.
Test Case Derivation: From FR 17 — PTs receive session reports automatically.
How Test Will Be Performed: Automated generation and field-validation of report content.

Test ID: RPT-UT-002
Type: Functional, Automatic
Initial State: Patient repeatedly fails exercise.
Input: Record 3 failed attempts.
Output: Alert appears in PT dashboard.
Test Case Derivation: From FR 20 — alerts for repeated incorrect attempts.
How Test Will Be Performed: Simulate consecutive failed submissions and verify dashboard flag.

6.3.7 Module 7: Data Security and Privacy

Ensures data protection as required by FR 21–23 and SR IR 1–SR PR 2.

Test ID: DATA-UT-001
Type: Functional, Automatic
Initial State: Database ready for new patient data.
Input: Save patient profile with personal and medical details.
Output: Data stored in encrypted form; readable only via authorized session.
Test Case Derivation: From FR 21 and SR-IR 1 — encryption at rest and in transit.
How Test Will Be Performed: Automated write-and-read test confirming ciphertext storage.

Test ID: DATA-UT-002
Type: Functional, Automatic
Initial State: Two patient accounts exist.
Input: Patient A attempts to access Patient B's records.
Output: Access denied; system logs unauthorized attempt.

Test Case Derivation: From FR 23 and SR-PR 1 — patients may access only their own data.
How Test Will Be Performed: Automated access attempt verifying access-control policies.

6.4 Tests for Nonfunctional Requirements

These tests focus on performance, robustness, scalability, and maintainability characteristics that affect the overall reliability of the system.

6.4.1 Module 8: Performance and Latency

Test ID: PERF-UT-001
Type: Dynamic, Automatic
Initial State: Application idle.
Input/Condition: Load exercise interface.
Output/Result: Interface loads within 2 seconds.
How Test Will Be Performed: Automated timing test using performance hooks.

Test ID: PERF-UT-002
Type: Dynamic, Automatic
Initial State: Running video analysis.
Input/Condition: Introduce form deviation.
Output/Result: Corrective feedback appears within 1 second.
How Test Will Be Performed: Mock time-stamped analysis results; measure delay to user prompt.

6.4.2 Module 9: Reliability and Offline Operation

Test ID: REL-UT-001
Type: Functional, Automatic
Initial State: Network unavailable.
Input/Condition: Attempt to upload exercise video.

Output/Result: Upload queued locally; auto-retries every 30 seconds until success.
How Test Will Be Performed: Network simulation tests queue persistence and retry logic.

Test ID: REL-UT-002
Type: Functional, Automatic
Initial State: Device offline.
Input/Condition: User opens exercise list.
Output/Result: Cached exercises and instructions load successfully; sync resumes when online.
How Test Will Be Performed: Offline simulation verifying cache and synchronization behavior.

6.4.3 Module 10: Security and Backup

Test ID: SEC-UT-001
Type: Functional, Automatic
Initial State: System with active data records.
Input/Condition: Trigger daily backup job.
Output/Result: Backup created and timestamped successfully.
How Test Will Be Performed: Automated scheduler simulation confirming backup creation.

Test ID: SEC-UT-002
Type: Functional, Automatic
Initial State: Idle session.
Input/Condition: User inactive for 60 minutes.
Output/Result: Session automatically logs out and prompts for re-authentication.
How Test Will Be Performed: Time-simulation test confirming session timeout.

6.4.4 Module 11: Scalability and Extensibility

Test ID: EXT-UT-001
Type: Functional, Automatic
Initial State: Base application loaded with knee module.
Input/Condition: Add new "shoulder" exercise definitions.
Output/Result: System loads new joint without modifying core logic.
How Test Will Be Performed: Add-on simulation validating modular design extensibility.

6.5 Traceability Between Test Cases and Modules

Module	Primary Requirements Covered	Representative Test IDs
1. Authentication & Account Management	FR 1–4; SR AR 1–4; SR3	AUTH-UT-001–003
2. Organization Linking & Access Codes	FR 1.2–1.3; FR 2–3	ORG-UT-001–002
3. Exercise Program Management	FR 5–6; FR 10–11; FR 22; FR 27	PROG-UT-001–003
4. Video Capture & Review	FR 7; FR 15; FR 26	CAP-UT-001–003
5. Computer Vision & Form Analysis	FR 8; FR 12–14; FR 16; PR-PAR 1–2; PR-SCR 1	CV-UT-001–003
6. Reporting & Dashboards	FR 17–20; FR 25	RPT-UT-001–002
7. Data Security & Privacy	FR 21–23; SR-IR 1; SR-PR 1–2	DATA-UT-001–002
8. Performance & Latency	PR-SL 1–2	PERF-UT-001–002
9. Reliability & Offline Operation	PR-RFT 2; MS-AD 1	REL-UT-001–002
10. Security & Backup	SR AR 3; MS-MNT 1	SEC-UT-001–002
11. Scalability & Extensibility	PR-CR 1; PR-SER 1	EXT-UT-001

Table 6: Traceability Between Test Cases and Modules

This matrix demonstrates coverage of all functional and nonfunctional requirements through at least one representative unit test per module.

7 Appendix

Additional project information to expand on the previous sections. Mostly example or sample information at this time.

7.1 Example script

Example script layout for system testing:
/*
Create user account testing
New account creation (user_acc_create)
Invalid format of field inputs (invalid_formatCreate)
Invalid sign up credentials (cannot_verify_creds)
Presence of an existing account (acc_exists)

PT focused testing
PT successfully sends invite to patient (PT_sends_invite)
PT adds/modified patient exercise (PT_adjusts_exercise)
PT deletes patient account (PT_deletes_patient_acc)

Patient focused testing
System_exercise_overview
Video_store
Video_retrieval
System_analysis_feedback
Post_ex_patient_feedback
Incorrect_form_adjustment

NFR testing
Ex_int_speed
Fail_recovery
Interrupted_vid
/

7.2 Usability Survey Questions

Advisor SRS Questions:

1. Are there any sections of the requirements which are found to be confusing or hard to measure?
2. Are there any sections of the system you feel the requirements do not expand on?
3. Are there any sections of the system you feel the requirements expand on that is out of scope?
4. Do you feel the requirements are extensive enough?
5. Do you feel the requirements are tracked well?

Design Questionnaire Questions:
"Strongly Disagree" to "Strongly Agree" style questioning.

1. The system navigation is intuitive.
2. The system's functions are all easily accessible.
3. The system helped set up your exercises for success.
4. The system enables you to complete your exercises with better guidance.
5. The system enables you to complete your exercises with better confidence.
6. The feedback returned during your exercise was helpful.
7. The overview data from your exercise was helpful.
8. You would you use this software for your exercises again.

Could have a final open-ended question based on user pain points with the system I.e. what did not work well for individuals.

Software Validation Questionnaire Questions:
"Strongly Disagree" to "Strongly Agree" style questioning.

1. This software assisted you in completing your exercises.
2. This software was easy to use.

3. This software was easy to navigate.
4. The feedback from the system assisted you in fixing your form.
5. The feedback from the system generated greater confidence in your form.
6. The feedback from the system was meaningful and helpful.
7. The data generated from your exercises was helpful.
8. The system responded how you expected it to during usage.
9. The system clearly described what was going on.
10. You felt adequately trained to use the system from the system dialogues.
11. You felt safe while using the system.

Ui_testing Survey:

1. How would you rate your overall experience with the application (1 - poor, 5 - excellent)?
2. How would you rate the visual appeal of the application (1 - poor, 5 - excellent)?
3. How easy was it to navigate through the application? (1 - difficult, 5 - easy)
    (a) Were the tutorials helpful? (1 - not helpful, 5 - very helpful)
    (b) How logical was the arrangement of the different components within the system (1 - very illogical, 5 - very logical)?
4. Which areas could use some improvement?
    (a) Navigation
    (b) Menu structure
    (c) Colour/contrast scheme
    (d) Error messages

(e) Other (With open text entry)

Patient Post-Exercise Feedback Form:

1. How did you feel before the exercise? (1 - good, 5-bad)
   (a) Did you have tightness? Yes/no
   (b) Did you have any pain? Yes/no.
      i. How bad was your pain? (1 - no pain, 10 - unbearable pain)
      ii. What was the perceived level of pain/difficulty? (1 - very easy, 10 - very difficult)
      iii. Where was your pain located? (Open text entry)
   (c) Have you noticed any recent changes? (Open text entry)
2. If you had any pain DURING the exercise, how bad was it? (1 - no pain, 10 - unbearable pain)
3. How did you feel after the exercise? (1 - good, 5-bad)
   (a) Did you have tightness? Yes/no
   (b) Did you have any pain? Yes/no.
      i. How bad was your pain? (1 - no pain, 10 - unbearable pain)
      ii. What was the perceived level of pain/difficulty? (1 - very easy, 10 - very difficult)
      iii. Where was your pain located? (Open text entry)
   (c) Have you noticed any recent changes? (Open text entry)
4. How challenging was the exercise today? (1 - easy, 5 - very hard)
   (a) Do you have any feedback for your physiotherapist? (Open text entry)
5. In your opinion, is this treatment plan helping your health? Yes/no. Choose the option that describes how the plan is helping (Can choose multiple).
   (a) My movement
   (b) My flexibility

(c) My pain

(d) My mood

6. In your opinion, how difficult is the current prescribed exercise plan? (1 - easy, 5 - very difficult)

   (a) Should your physiotherapist change the difficulty of your plan? (1 - no change, 5 - Large increase in difficulty)

References

Eman Ashraf, Cieran Diebolt, Matthew Hyunh, Vaisnavi Shantamoorthy, and Maham Siddiqui. Design thinking document. https://github.com/M9Huynh/technically-functional/tree/32182cfc56c602466e270b7fcbad01d892911ea1/docs/Extras/DesignThinking, 2025a.

Eman Ashraf, Cieran Diebolt, Matthew Hyunh, Vaisnavi Shantamoorthy, and Maham Siddiqui. Development plan. https://github.com/M9Huynh/technically-functional/blob/32182cfc56c602466e270b7fcbad01d892911ea1/docs/DevelopmentPlan/DevelopmentPlan.pdf, 2025b.

Eman Ashraf, Cieran Diebolt, Matthew Hyunh, Vaisnavi Shantamoorthy, and Maham Siddiqui. Hazard analysis. https://github.com/M9Huynh/technically-functional/blob/32182cfc56c602466e270b7fcbad01d892911ea1/docs/HazardAnalysis/HazardAnalysis.pdf, 2025c.

Eman Ashraf, Cieran Diebolt, Matthew Hyunh, Vaisnavi Shantamoorthy, and Maham Siddiqui. Problem statement and goals. https://github.com/M9Huynh/technically-functional/blob/32182cfc56c602466e270b7fcbad01d892911ea1/docs/ProblemStatementAndGoals/ProblemStatement.pdf, 2025d.

Eman Ashraf, Cieran Diebolt, Matthew Hyunh, Vaisnavi Shantamoorthy, and Maham Siddiqui. System requirements specification. https://github.com/M9Huynh/technically-functional/blob/32182cfc56c602466e270b7fcbad01d892911ea1/docs/SRS/SRS.pdf, 2025e.

Eman Ashraf, Cieran Diebolt, Matthew Hyunh, Vaisnavi Shantamoorthy, and Maham Siddiqui. User instructional video. https://github.com/M9Huynh/technically-functional/tree/32182cfc56c602466e270b7fcbad01d892911ea1/docs/Extras/UserInstructionalVideo, 2025f.

Appendix — Reflection

The information in this section will be used to evaluate the team members on the graduate attribute of Lifelong Learning.

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. What went well while writing this deliverable?

2. What pain points did you experience during this deliverable, and how did you resolve them?

3. What knowledge and skills will the team collectively need to acquire to successfully complete the verification and validation of your project? Examples of possible knowledge and skills include dynamic testing knowledge, static testing knowledge, specific tool usage, Valgrind etc. You should look to identify at least one item for each team member.

4. For each of the knowledge areas and skills identified in the previous question, what are at least two approaches to acquiring the knowledge or mastering the skill? Of the identified approaches, which will each team member pursue, and why did they make this choice?

Vaisnavi - Reflection

1. I think one main thing that went well while writing this deliverable was our communication as a group and how that played into the division of work. We had two sub-teams where Matthew and I were focusing

on the POC, while Cieran, Maham and Eman were focused on this VnV deliverable. Our communication throughout aided in making this process of working on the deliverable seamless as everyone seemed to be very looped in on what each sub-team was working on. We had our weekly meetings to touch base and ask for any help if it was needed, and it worked very well.

2. I think one of the main pain points we came across was as a result of us working on the POC, it was a little bit hard at the start to fully keep up with updates for the VnV Plan. However, as mentioned above, with sharing our progress and updates during our team meetings, this helped in providing more clarity and consistency between all group members, and it allowed everyone to be on the same page.

3. **GROUP Q**

4. To improve the knowledge of OpenCV and MediaPipe pose detection for accurate testing of body landmark tracing, we could do the two approaches:

   (a) Follow documentation and existing examples to understand how to properly utilize OpenCV and Mediapipe to accurately detect the varying body landmarks through pose detection.

   (b) Try to work with small test cases and work on small files to test it out on different body landmarks and see how it works.

Maham - Reflection

1. The team was split into two groups: one was responsible for the POC demonstration, and the other was completing the VnV report. All members were in constant communication even though they had other (very) important deadlines to meet. The sub-teams functioned very well, because each team had members who were (somewhat) more comfortable in the area they were responsible for, and others that wanted to expand their skillset. There was a good balance present. In the end, all members delivered work of the highest calibre and I am happy to be a part of such a great team.

2. I do not think it was a pain point but I had underestimated the amount of thought and importance that my portion would entail. I think it was challenging balancing the high priority of this deliverable (my part) and my other important (health) engagements as well. However, I used all the time management skills that I have learned and effectively managed to chunk my work into more realizable chunks while ensuring I am taking care of my well-being and delivered some good quality work.

3. **GROUP Q**

4. I had previously acquired some knowledge on software testing and have always been fascinated by it. But to really master the application, I had to be very detail oriented. I did a deep dive into the various components of our system. However, I realized that I had to be careful and not make my focus too narrow - because I was developing system tests. I had to look at the overall picture. I did this in 2 ways. Firstly, I researched 'system testing' to further increase my knowledge and get a more concrete idea. However, since I had prior knowledge, I found most of the resources online just broke down system testing into different unit tests. I did not want to pursue that because Eman was working on the unit testing component. I spoke to my teammates on the possible overlap between unit and system testing, and was more confident in my (potential) analysis and approach. In the end, I refined my testing knowledge and by designing different test cases for any possible scenario, and making sure the system was approached from various aspects without being too focused on individual modules.

Matthew - Reflection

1. I think one thing that went well when writing this deliverable was how we divided these two weeks up. We had taken the advice from the professor to begin development on the POC while working on the VnV plan. The team was divided into myself and Vais on the POC while Cieran, Maham and Eman were working on the VnV plan. This allowed us to do brief check-ins with each other while beginning development and thus more time to develop before the POC demo.

2. I think some pain points during this deliverable was tied to external workloads. Since half of the deliverable encompassed reading week, this

meant that the team had midterms and assignments for other classes due before this deliverable and managing that was more difficult. We resolved this by being transparent with each of our workloads, by letting each other know in our Teams chat if we were going to be busy and when the next time we would be available is. This allowed us to plan the deliverable better and achieve internal deadlines.

3. **GROUP Q**

4. I would like to improve my knowledge of stakeholder understanding. I think that with stakeholder understanding it is important as they are the target audience for the application and their needs are to be met even if it differs from our own since they are the target. My approach for that will consist of understanding interviews as well as conversations with the stakeholder during testing. Another approach will be different user groups to see if our initial persona and ideas for the stakeholder would make sense.
   I think the is important since it is easy to develop requirements by making assumptions but to bridge the gap to the target audience is crucial to make a product or service that they would actually use.

Cieran - Reflection

1. Team communication as always but I won't expand further on that. There was a decision to split the team into two subteams; a proof of concept team to begin the implementation and a verification and validation team to tackle this document. I feel the subteams have both been incredibly efficient in their work and the communication between the two has not faltered. The proof of concept team worked well together in overall and in subteam meetings to grasp an understanding of the software packages we will be utilizing while the verification and validation team ensured the entire system was covered within the tests outlined in this document.

2. I expanded on it during the last deliverable, that each person had this 'own version' of the system in their mind and we were all building towards a different thing when the assignment started. That wasn't entirely gone here and we hit a few bumps worrying about what each of the others was including within the system while thinking we don't need

that to be included. Through team meetings and a general walkthrough of the flow of the program, we all got on the same page and managed to generate tests and feel as if we've properly outlined a system to ensure the system is designed properly.

3. **GROUP Q**

4. I feel my personal interaction with test suite development lacks for the requirements of this project. To advance this I will attempt to do the following:

   (a) Follow online guides, walkthroughs, and documentation on the software packages (Python Extension, pytest, pylint, etc.) to ensure the software testing suite developed contains both meaningful tests derived from this document while also covering all the software.

   (b) Generate and execute tests in advancing complexity to ensure an understanding of the routines gone through with a testing suite. 'Getting my hands dirty'.

Eman - Reflection

1. The team worked together effectively by dividing into two focused groups, one for the POC demonstration and another for the VnV plan. Communication was strong throughout, and everyone contributed their strengths while helping others learn. The collaboration created a good balance where team members could build on their existing skills while developing new ones.

2. My main challenge was ensuring complete test coverage across all system modules. I initially underestimated how much detail was needed to properly test each component in isolation. Identifying all the edge cases and understanding how to structure effective unit tests took more effort than expected. I resolved this by creating a systematic checklist that mapped each module to its requirements, then worked through them methodically. Collaborating with teammates to review my test cases also helped catch gaps I had missed.

3. **GROUP Q**

4. I focused on mastering unit testing and test coverage. My first approach was working through practical examples to understand fixtures, mocking, and test isolation. My second approach was carefully reviewing our system requirements and architecture to understand what each module should do. I chose this combination because I needed both the technical testing skills and domain knowledge about our system. Understanding the requirements helped me design tests that actually validated meaningful functionality rather than just checking basic operations. This dual approach allowed me to create comprehensive unit tests covering all eleven modules in our system.